Data Mining

Week 10

Submitter - Himanshu Singh

Fraud Waste Abuse Detection in Healthcare Insurance - Project Milestone 3

Model Building and Evaluation

Milestone 1 **********

Problem Statement

The primary objective of this project is to develop a predictive model to identify Fraud, Waste, and Abuse (FWA) among healthcare providers. By analyzing claims data, the project aims to "Predict potentially fraudulent providers" while uncovering the most significant variables that signal suspicious behavior. This intelligence allows organizations to flag high-risk claims for in-depth investigation, ensuring that resources are directed toward the most likely instances of financial leakage.

Problem Description

Healthcare insurance is a vital pillar of social stability, protecting citizens from the financial hardships of medical uncertainty. However, the integrity of both public and private insurance schemes is constantly threatened by Fraud, Waste, and Abuse (FWA):

Fraud: Intentional deception or misrepresentation, such as billing for services never rendered or "phantom" patients. Waste: Overutilization of services or practices that result in unnecessary costs, such as performing excessive diagnostic tests that are not medically required. Abuse: Actions that are inconsistent with sound fiscal, business, or medical practices, resulting in improper payment (e.g., upcoding or unbundling procedures).

FWA is often an organized crime involving collusion between providers, physicians, and beneficiaries. Specifically, Provider Fraud is a massive burden on Medicare; for instance, providers may exaggerate billing amounts or facilitate benefits for unauthorized individuals.

The cost of misidentification is high: failing to detect FWA leads to substantial financial losses, while incorrectly flagging legitimate providers (False Positives) causes administrative friction and delays essential patient care. A precise, data-driven approach is required to balance aggressive cost recovery with the smooth adjudication of valid claims.

Source of data: Elsevier''s Mendeley Data repository is a participating member of the National Institutes of Health (NIH) Office of Data Science Strategy (ODSS) GREI project. liang, tim (2024), “HEALTHCARE PROVIDER FRAUD DETECTION ANALYSIS”, Mendeley Data, V1, doi: 10.17632/gsn2hyty37.1 https://data.mendeley.com/datasets/gsn2hyty37/1

In [164…

```
import pandas as pd
import os
import numpy as np
import matplotlib.pyplot as plt
import numpy as np
```

```python
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')
from tqdm import tqdm
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.model_selection import GridSearchCV
from sklearn import metrics
from sklearn.metrics import roc_curve, auc , f1_score
from sklearn.metrics import confusion_matrix
```

In [165…

```python
data = pd.read_csv('Train.csv')
data_beneficiary = pd.read_csv('Train_Beneficiarydata.csv')
data_inpatient = pd.read_csv('Train_Inpatientdata.csv')
data_outpatient = pd.read_csv('Train_Outpatientdata.csv')
```

Exploratory Data Analysis

We will try to perform EDA (Exploratory Data Analysis) on the dataset we have

- Diagnosis Codes (primarily ICD-10 codes) are the standardized alphanumeric strings used to describe a patient's medical condition, symptoms, or injuries.
- Procedure Codes are standardized numeric or alphanumeric codes used to identify the specific medical, surgical, or diagnostic services provided to a patient. While Diagnosis Codes explain "why" the patient is there, Procedure Codes explain "what" was done to them.
- Reimbursement of the Claim Amount.
- Labelled data of Potential Fraud Providers.
- Fraud case with respect to Labelled Fraud Provider data.
- Money Lost in due to Fraud Cases.

In [166…

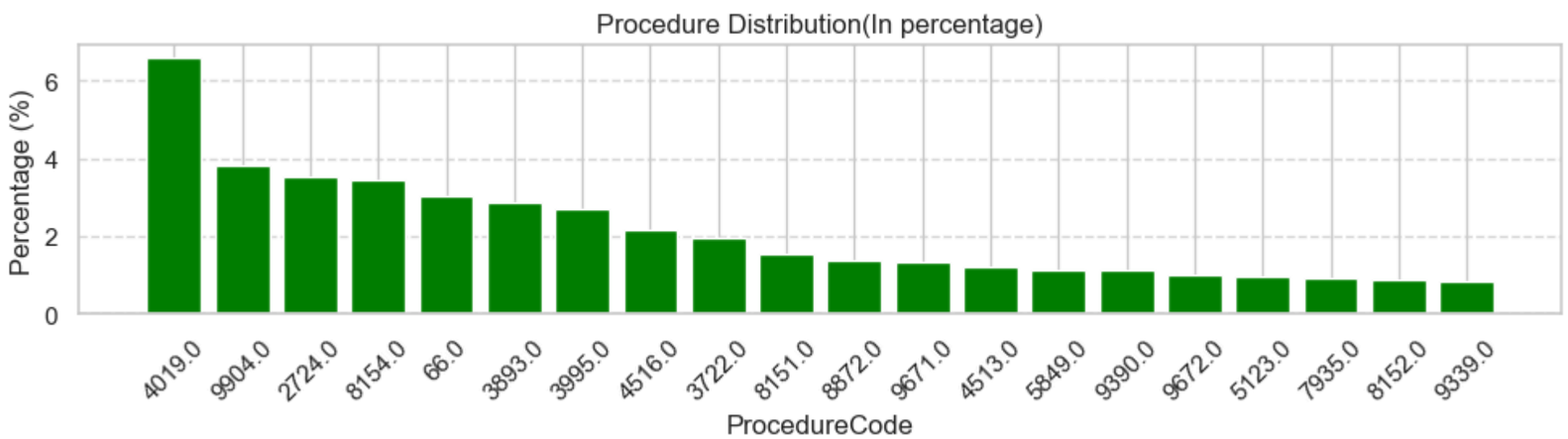
```python
def get_year(date):
    """This function returns year from date"""
    date = str(date)
    return(int(date[:4]))

def get_month(date):
    """This function returns month from date"""
    date = str(date)
    return (int(date[5:7]))
# Function to analyse categorical columns

def analyse_cat_columns(dataset, col_to_analyse, prefix='', title='Distribution in percentage',
                        top_val=30, y_lim=None, color='green'):
    """Plots a bar graph for the percentage distribution of a categorical column."""

    # Calculate counts and convert to percentage in one step
    counts = dataset[col_to_analyse].value_counts().head(top_val)
    percentages = (counts / len(dataset)) * 100

    # Create the plot labels with the optional prefix
    labels = [f"{prefix}{label}" for label in percentages.index]

    # Plotting
    plt.figure(figsize=(10, 3))
    plt.bar(labels, percentages.values, color=color)

    # Formatting
```

```
    plt.title(title)
    plt.ylabel('Percentage (%)')
    plt.xlabel(col_to_analyse)
    plt.xticks(rotation=45)
    plt.grid(axis='y', linestyle='--', alpha=0.7)

    if y_lim is not None:
        plt.yticks(y_lim)

    plt.tight_layout()
    plt.show()
```

Inpatient Data Analysis

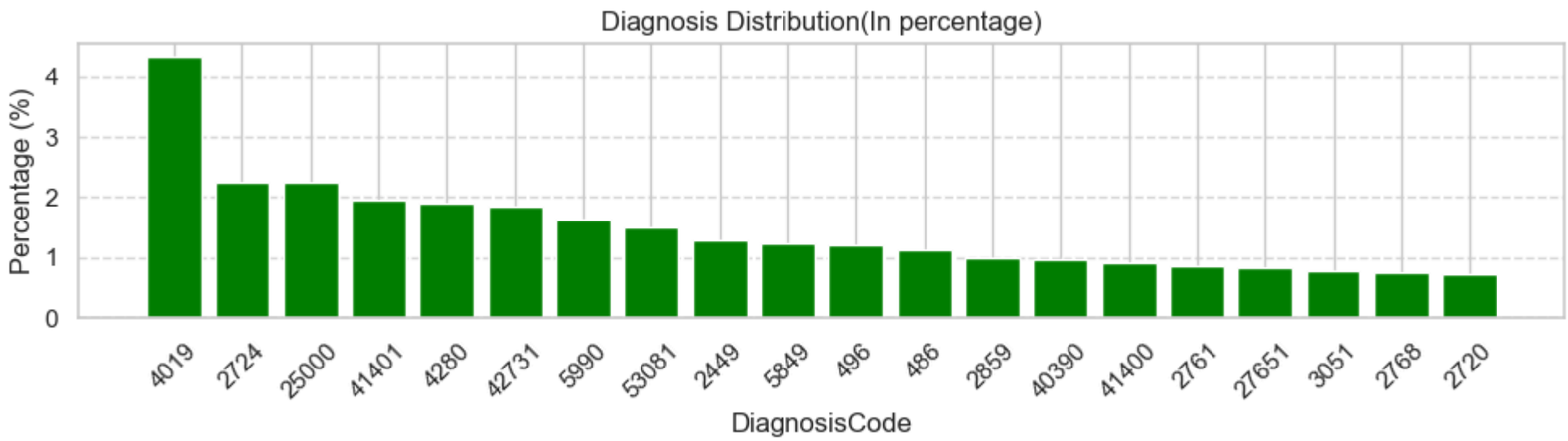
In [167…

```
# Select all columns that start with 'ClmProcedureCode'
proc_cols = [col for col in data_inpatient.columns if 'ClmProcedureCode' in col]

# Concatenate them all and drop NaNs
inpatient_df = pd.DataFrame(
    pd.concat([data_inpatient[col] for col in proc_cols], axis=0).dropna(),
    columns=['ProcedureCode']
)
analyse_cat_columns(inpatient_df,col_to_analyse='ProcedureCode' ,title = 'Procedure Distribution(In percentage)',top_val = 20)
```



In [168…

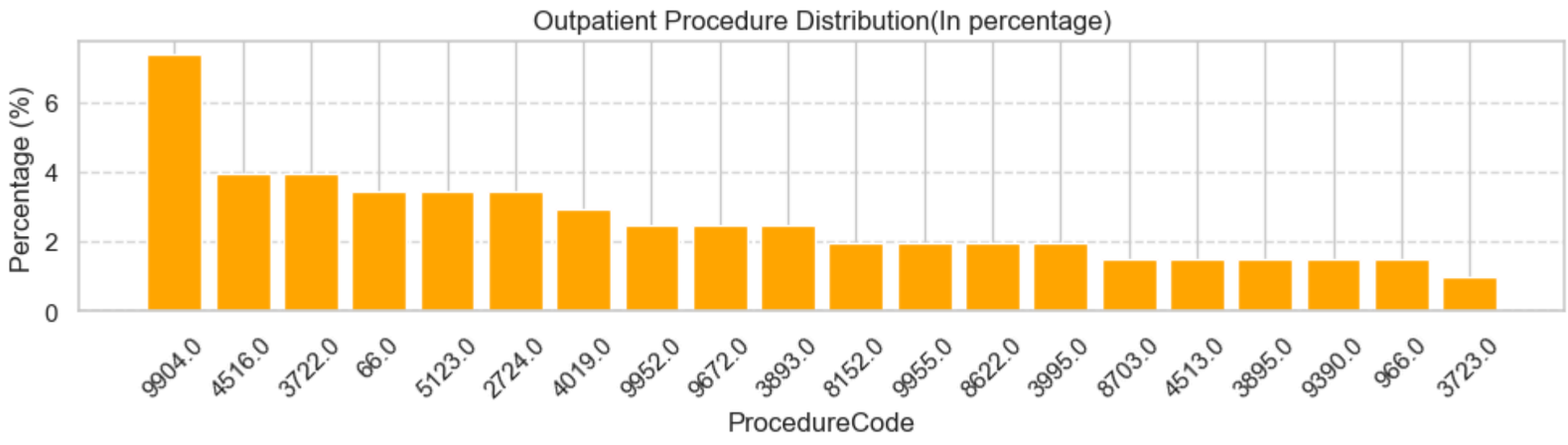
```
# Automatically grabs ClmDiagnosisCode_1 through ClmDiagnosisCode_10
diag_cols = data_inpatient.filter(like='ClmDiagnosisCode').columns

# Concatenate into a single Series and remove missing values
inpatient_diag_df = pd.DataFrame(
    pd.concat([data_inpatient[col] for col in diag_cols], axis=0).dropna(),
    columns=['DiagnosisCode']
)
analyse_cat_columns(inpatient_diag_df,col_to_analyse='DiagnosisCode' , title = 'Diagnosis Distribution(In percentage)',top_val = 20, color ='green')
```



Outpatient Data Analysis

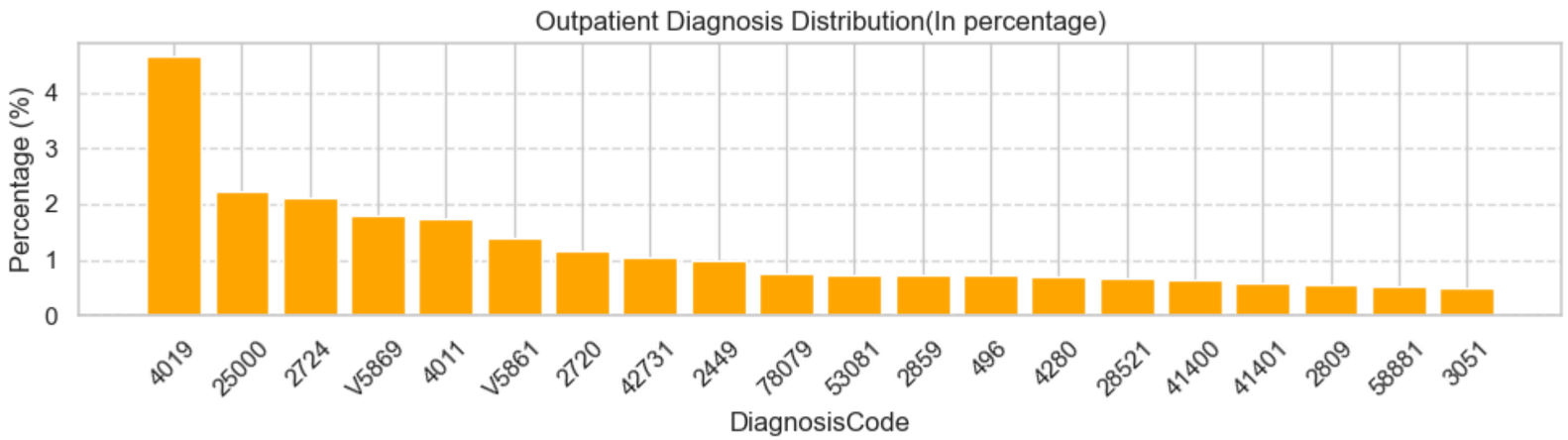
In [169…

```
# Select all columns that start with 'ClmProcedureCode'
proc_cols = [col for col in data_outpatient.columns if 'ClmProcedureCode' in col]

# Concatenate them all and drop NaNs
outpatient_df = pd.DataFrame(
    pd.concat([data_outpatient[col] for col in proc_cols], axis=0).dropna(),
    columns=['ProcedureCode']
)
analyse_cat_columns(outpatient_df,col_to_analyse='ProcedureCode' ,title = 'Outpatient Procedure Distribution(In percentage)',top_val = 20, color='orange')
```



In [170…

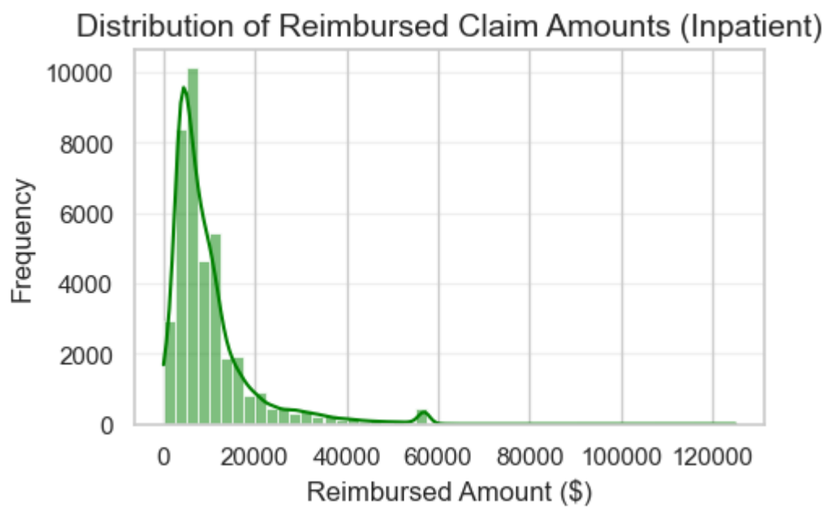
```
# Automatically grabs ClmDiagnosisCode_1 through ClmDiagnosisCode_10
diag_cols = data_outpatient.filter(like='ClmDiagnosisCode').columns

# Concatenate into a single Series and remove missing values
outpatient_diag_df = pd.DataFrame(
    pd.concat([data_outpatient[col] for col in diag_cols], axis=0).dropna(),
    columns=['DiagnosisCode']
)
analyse_cat_columns(outpatient_diag_df,col_to_analyse='DiagnosisCode' , title = 'Outpatient Diagnosis Distribution(In percentage)',top_val = 20, color='orange')
```



Analysis of Claim Amount Reimbursed

In [171…

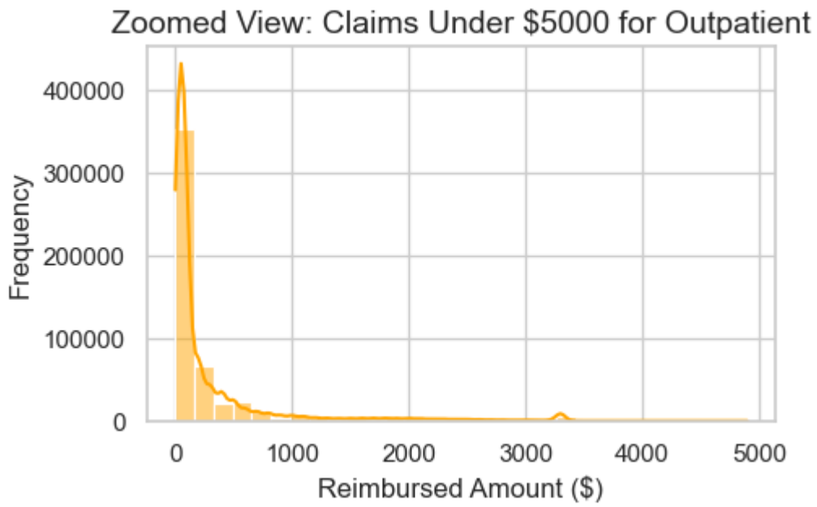
```
# Checking distribution of Claim amount for Inpatient Claims

plt.figure(figsize=(5, 3))
# Using histplot for a more modern, detailed view
sns.histplot(data_inpatient['InscClaimAmtReimbursed'], kde=True, color='green', bins=50)

plt.title('Distribution of Reimbursed Claim Amounts (Inpatient)', fontsize=14)
plt.xlabel('Reimbursed Amount ($)', fontsize=12)
plt.ylabel('Frequency', fontsize=12)
plt.grid(axis='y', alpha=0.3)
plt.show()
```



In [172…

```
# Checking distribution of Claim amount for Outpatient Claims
# Create a temporary view of only small claims
small_claims = data_outpatient[data_outpatient['InscClaimAmtReimbursed'] < 5000]

plt.figure(figsize=(5, 3))
sns.histplot(small_claims['InscClaimAmtReimbursed'], kde=True, color='orange', bins=30)
```

```
plt.title('Zoomed View: Claims Under $5000 for Outpatient', fontsize=14)
plt.xlabel('Reimbursed Amount ($)')
plt.ylabel('Frequency')
plt.show()
```



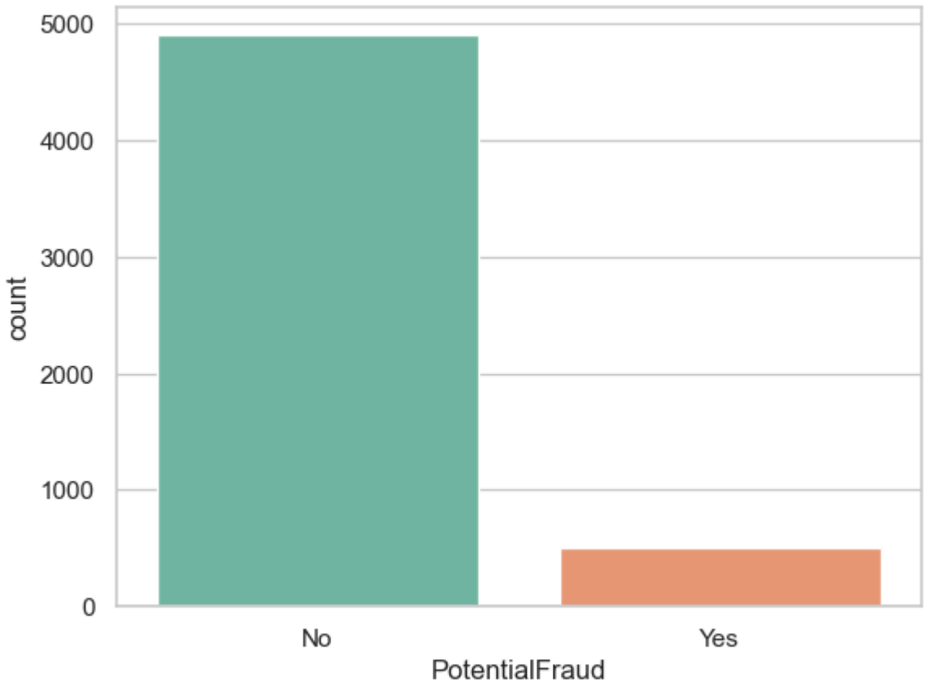
Analysis of Labeled data for Fraud Providers

In [173…
```
#check all the columns of the labeled  dataset and top 5 data points
print('Columns in this dataset are : ')
print(data.columns)
```

```
Columns in this dataset are :
Index(['Provider', 'PotentialFraud'], dtype='object')
```

In [174…
```
# Check the distribution of the target variable
sns.countplot(x='PotentialFraud',data=data, palette='Set2')
```

Out[174…
```
<Axes: xlabel='PotentialFraud', ylabel='count'>
```



Percentage of fraudulent encounters Inpatient / Outpatient

In [175…

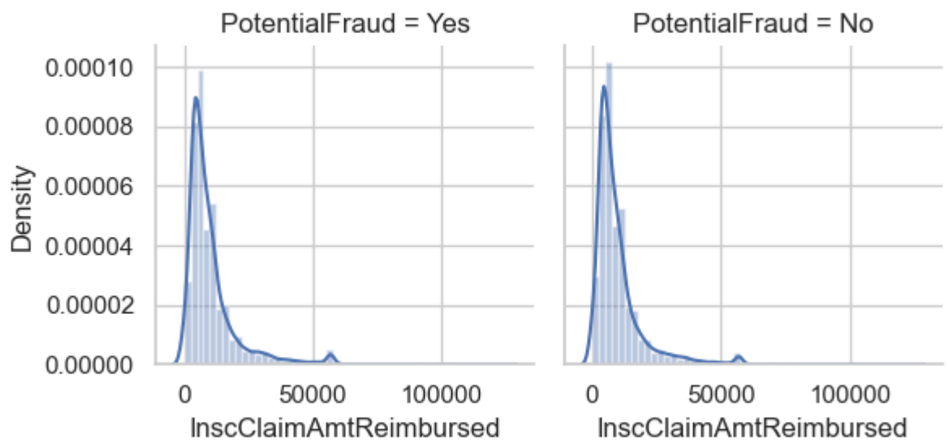
```
# Now we will filter out only the fraudulent providers to analyze their characteristics in more detail.

only_fraud_provider = data.loc[data['PotentialFraud']=='Yes']
fraud_provider_inpatient_df = pd.merge(data_inpatient, only_fraud_provider, how='inner', on='Provider')
print("Percentage of fraud cases related to inpatient data : ",(fraud_provider_inpatient_df.shape[0]/data_inpatient.shape[0])*100)

fraud_provider_outpatient_df = pd.merge(data_outpatient, only_fraud_provider, how='inner', on='Provider')
print("Percentage of fraud cases related to outpatient data : ",(fraud_provider_outpatient_df.shape[0]/data_outpatient.shape[0])*100)
```

```
Percentage of fraud cases related to inpatient data :  57.81983495577408
Percentage of fraud cases related to outpatient data :  36.58112130290089
```

Calculating Money lost in Fraud for both Inpatient and outpatient data

In [176…

```
# Now we will merge the beneficiary dataset with the inpatient dataset to analyze the characteristics of beneficiaries related to
# fraudulent providers in more detail.
inpat_labeled_provider_df = pd.merge(data_inpatient , data , how='inner', on='Provider')
sns.FacetGrid(inpat_labeled_provider_df, col='PotentialFraud',height=3).map(sns.distplot, "InscClaimAmtReimbursed",).add_legend()
plt.show()
plt.tight_layout()
Total_money_lost = inpat_labeled_provider_df.loc[inpat_labeled_provider_df['PotentialFraud']== 'Yes'].InscClaimAmtReimbursed.sum()
print("Total money lost : ",Total_money_lost)
```




```
Total money lost :  241288510
<Figure size 640x480 with 0 Axes>
```

In [177…

```
# calculating total money lost outpatient fraudulent encounters
outpat_labeled_provider_df = pd.merge(data_outpatient , data , how='inner', on='Provider')
Total_money_lost = outpat_labeled_provider_df.loc[outpat_labeled_provider_df['PotentialFraud']== 'Yes'].InscClaimAmtReimbursed.sum()
print("Total money lost : ",Total_money_lost)
```

```
Total money lost :  54392610
```

In [178…

```
print('Total money lost as per the data = ','241,288,510 + 54,392,610, That is around 295 Million' )
```

```
Total money lost as per the data =  241,288,510 + 54,392,610, That is around 295 Million
```

Conclusion

We found about Procedure code 4019 is the most common procedure and Diagnosis code 4019 that is followed for inpatient.

We found about Procedure code 9904 is the most common procedure and Diagnosis code 4019 that is followed for outpatient.

Almost all Reimbursed amount is between 0 and 20000 for Inpatient.

Labelled data is imbalanced.

Around 58% of the inpatient data for training has fraudulent encounter.

Around 37% of the outpatient data for training has fraudulent encounter.

Total money lost as per the data is = 241288510+54392610, That is around 290 Million

Till this point we have done thorough EDA on columns of each of the data files. Now we will merge them all and then perform EDA to establish a trend in data and the target.

Milestone 2 **********

Term Project Milestone 2: Data Preparation

Here is a list of steps to consider performing in Milestone 2:

- Drop any features that are not useful for your model building and explain why they are not useful.
- Perform any data extraction/selection steps.
- Transform features if necessary.
- Engineer new useful features.
- Deal with missing data (do not just drop rows or columns without justifying this).
- Create dummy variables if necessary.

We need to complete some more EDA for the project

Data preprocessing and Feature Engineering

In [179…

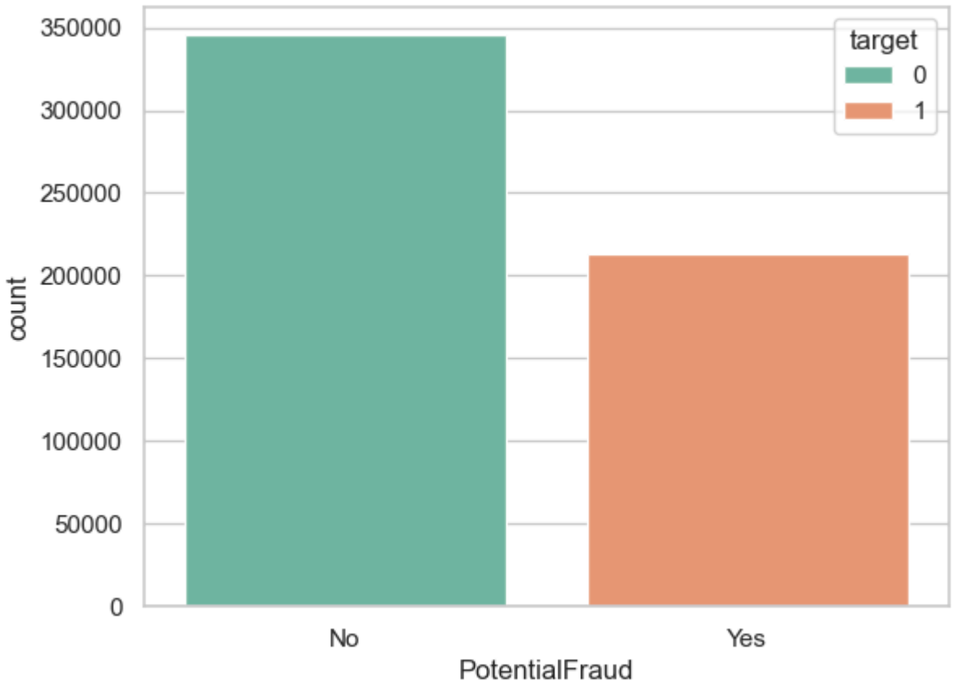
```
# Create a binary target variable for whether the claim was admitted (inpatient) or not (outpatient)

data_inpatient['is_admitted'] = 1
data_outpatient['is_admitted'] = 0

# Merge in_pt, out_pt and ben df into a single patient dataset
data1 = pd.merge(data_inpatient, data_outpatient,
                    left_on = [ idx for idx in data_outpatient.columns if idx in data_inpatient.columns],
                    right_on = [ idx for idx in data_outpatient.columns if idx in data_inpatient.columns],
                    how = 'outer').\
          merge(data_beneficiary,left_on='BeneID',right_on='BeneID',how='inner')

# Replace values with a binary annotation
data1 = data1.replace({'ChronicCond_Alzheimer': 2, 'ChronicCond_Heartfailure': 2, 'ChronicCond_KidneyDisease': 2,
                   'ChronicCond_Cancer': 2, 'ChronicCond_ObstrPulmonary': 2, 'ChronicCond_Depression': 2,
                   'ChronicCond_Diabetes': 2, 'ChronicCond_IschemicHeart': 2, 'ChronicCond_Osteoporasis': 2,
                   'ChronicCond_rheumatoidarthritis': 2, 'ChronicCond_stroke': 2, 'Gender': 2 },
                  0)
# Replace values with a binary annotation
data1 = data1.replace({'RenalDiseaseIndicator': 'Y'}, 1).astype({'RenalDiseaseIndicator': 'int64'})#array([0, 1], dtype=int64)
#merging the dataset created in above step with data
data_final = pd.merge(data1, data , how = 'outer', on = 'Provider' )
# adding extra column target, having binary annotation
data_final['target'] = data_final['PotentialFraud']

# representing potential fraud and 0 representing non potential fraud
data_final['target'] = data_final['target'].map({"Yes":1,"No":0})
data_final['target'] = data_final['target'].astype('category',copy=False)
```

In [180…

```
# checking the distribution of target variable in the final dataset

print(data_final.PotentialFraud.value_counts())
sns.countplot(x='PotentialFraud', data=data_final, palette='Set2', hue='target')
```

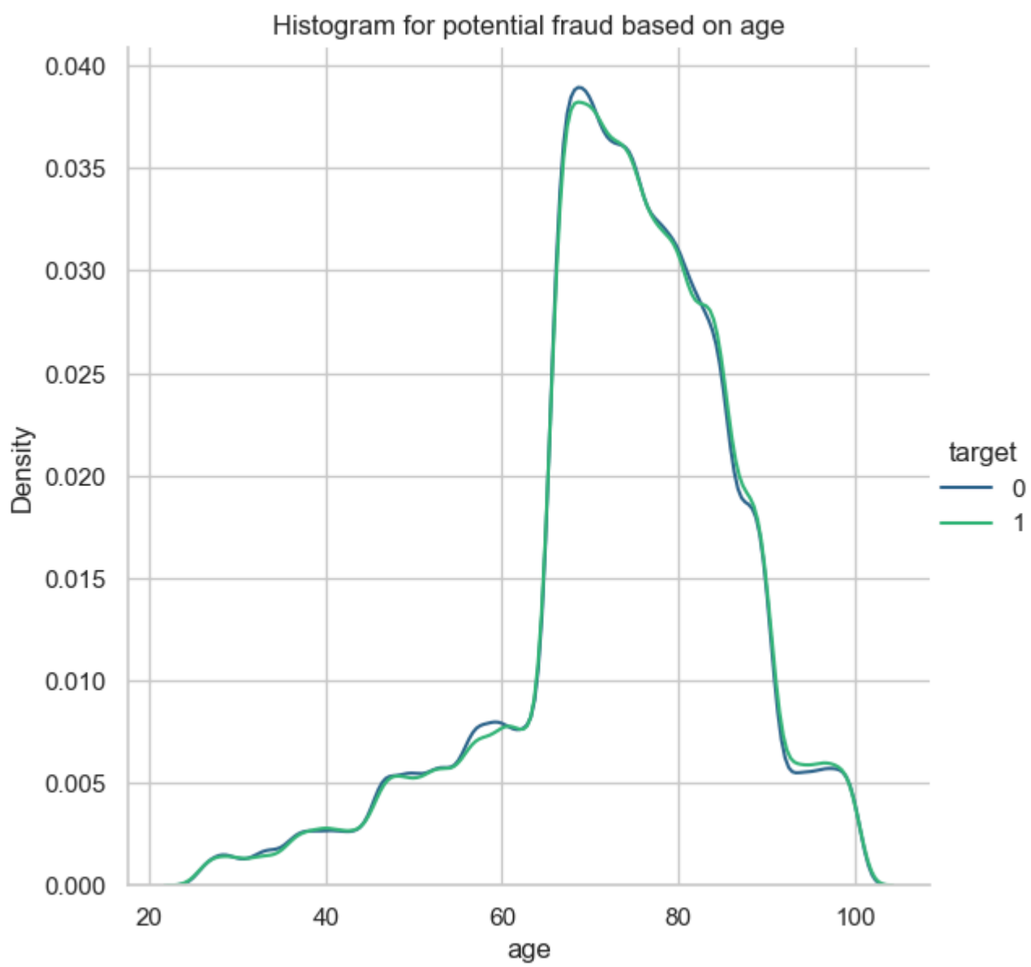
```
PotentialFraud
No     345415
Yes    212796
Name: count, dtype: int64
```

Out[180… <Axes: xlabel='PotentialFraud', ylabel='count'>



In [181…

```
# #Adding a column is_dead  = 1  DOD is given else is_dead = 0
# data_final.loc[data_final['DOD'].isnull(), 'Is_Dead'] = '0'
# data_final.loc[(data_final['DOD'].notnull()), 'Is_Dead'] = '1'

data_final['DOB'] =  pd.to_datetime(data_final['DOB'], format='%Y-%m-%d')
data_final['ClaimStartDt'] = pd.to_datetime(data_final['ClaimStartDt'], format='%Y-%m-%d')

#now = pd.to_datetime('2020-09-01' , format = '%Y-%m-%d')
data_final['DOB'] = data_final['DOB'].where(data_final['DOB'] < data_final['ClaimStartDt'])
data_final['age'] = (data_final['ClaimStartDt'] - data_final['DOB']).dt.days / 365.25
#data_final['age'] = (data_final['ClaimStartDt'] - data_final['DOB']) / np.timedelta64(1, 'Y')
#data_final['age'] = (data_final['ClaimStartDt'] - data_final['DOB']).astype('<m8[Y]')

#plotting age
sns.FacetGrid(data_final, hue="target" , height=6, palette='viridis').map(sns.kdeplot, "age",).add_legend()
plt.title('Histogram for potential fraud based on age')
plt.show()
plt.tight_layout()
```



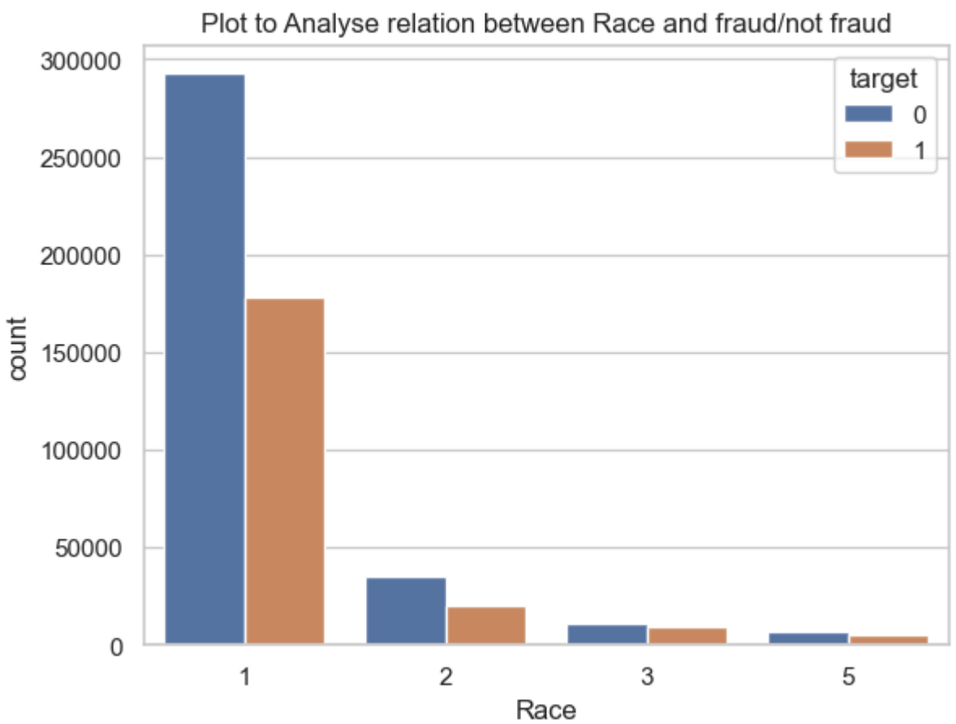

```
<Figure size 640x480 with 0 Axes>
```

In [182…

```
# Now we will analyze the relationship between Race and potential fraud cases in the dataset.
ax = sns.countplot(x='Race',data=data_final,hue='target')
ax.set_title("Plot to Analyse relation between Race and fraud/not fraud")
```

Out[182…

```
Text(0.5, 1.0, 'Plot to Analyse relation between Race and fraud/not fraud')
```



Conclusion

It is very important to see that most fraudulent cases is with patients belonging to a particular Race and that is labelled as 1

In [183…
```
# # Since we didn't have any use for DOD and DOB columns in our analysis, we will drop these columns from the final dataset.
data_final = data_final.drop(['DOD'], axis = 1)
data_final = data_final.drop(['DOB'], axis = 1)
```

In [184…
```
#calculating number of days a patient was admitted in the hospital and then removing admit and discharge date
#outpatients dont get admitted so we keep 0 for them
data_final['AdmissionDt'] = pd.to_datetime(data_final['AdmissionDt'] , format = '%Y-%m-%d')
data_final['DischargeDt'] = pd.to_datetime(data_final['DischargeDt'],format = '%Y-%m-%d')
data_final['Num_admit_days'] = ((data_final['DischargeDt'] - data_final['AdmissionDt']).dt.days)+1
data_final.loc[data_final['is_admitted'] == 0, 'Num_admit_days'] = '0'

data_final = data_final.drop(['DischargeDt'], axis = 1)
data_final = data_final.drop(['AdmissionDt'], axis = 1)
```

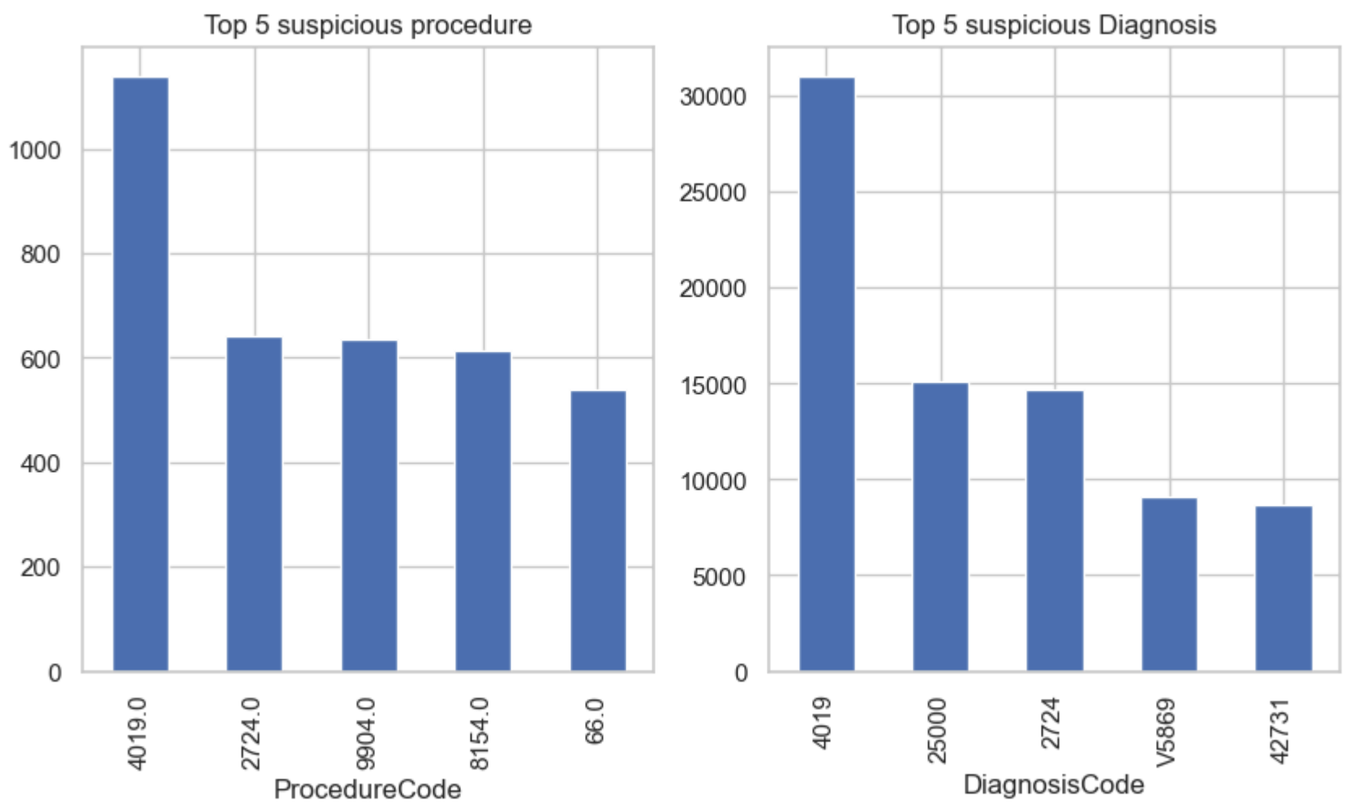
In [185…
```
#replacing NaN value with 0
data_final.loc[data_final['DeductibleAmtPaid'].isnull(),'DeductibleAmtPaid'] = 0
```

In [186…
```
# Now we will analyze the top procedure codes and diagnosis codes associated with potential fraud cases in the dataset.
# We will concatenate all the procedure code columns and diagnosis code columns into two separate dataframes
# and then plot the top 5 procedure codes and diagnosis codes associated with potential fraud cases in the dataset.

potential_fraud_df =data_final.loc[data_final['PotentialFraud'] == 'Yes']

df1 = pd.DataFrame(columns = ['ProcedureCode'])
df1['ProcedureCode'] = pd.concat([potential_fraud_df["ClmProcedureCode_1"],
```

```
                                        potential_fraud_df["ClmProcedureCode_2"],
                                        potential_fraud_df["ClmProcedureCode_3"],
                                        potential_fraud_df["ClmProcedureCode_4"],
                                        potential_fraud_df["ClmProcedureCode_5"],
                                        potential_fraud_df["ClmProcedureCode_6"]], axis=0, sort=True).dropna()

df2 = pd.DataFrame(columns = ['DiagnosisCode'])
df2['DiagnosisCode'] = pd.concat([potential_fraud_df["ClmDiagnosisCode_1"],
                                        potential_fraud_df["ClmDiagnosisCode_2"],
                                        potential_fraud_df["ClmDiagnosisCode_3"],
                                        potential_fraud_df["ClmDiagnosisCode_4"],
                                        potential_fraud_df["ClmDiagnosisCode_5"],
                                        potential_fraud_df["ClmDiagnosisCode_6"],
                                        potential_fraud_df["ClmDiagnosisCode_7"],
                                        potential_fraud_df["ClmDiagnosisCode_8"],
                                        potential_fraud_df["ClmDiagnosisCode_9"],
                                        potential_fraud_df["ClmDiagnosisCode_10"]], axis=0, sort=True).dropna()

plt.figure(figsize=(10, 5))
plt.subplot(121)
df1.ProcedureCode.value_counts().head(5).plot(kind = 'bar' , title ='Top 5 suspicious procedure')

plt.subplot(122)
df2.DiagnosisCode.value_counts().head(5).plot(kind = 'bar' , title ='Top 5 suspicious Diagnosis')
```

Out[186…

```
<Axes: title={'center': 'Top 5 suspicious Diagnosis'}, xlabel='DiagnosisCode'>
```



In [187…

```
# Defining a list of all diagnosis and procedure code columns to replace NaN values with 0

Diag_proce_col = ['ClmAdmitDiagnosisCode', 'ClmDiagnosisCode_1', 'ClmDiagnosisCode_10',
```

```
                  'ClmDiagnosisCode_2', 'ClmDiagnosisCode_3', 'ClmDiagnosisCode_4',
                  'ClmDiagnosisCode_5', 'ClmDiagnosisCode_6', 'ClmDiagnosisCode_7',
                  'ClmDiagnosisCode_8', 'ClmDiagnosisCode_9', 'ClmProcedureCode_1',
                  'ClmProcedureCode_2', 'ClmProcedureCode_3', 'ClmProcedureCode_4',
                  'ClmProcedureCode_5', 'ClmProcedureCode_6']

data_final[Diag_proce_col]= data_final[Diag_proce_col].replace({np.nan:0})
```

In [188…

```
# Now we will create new features based on the top 5 procedure codes and diagnosis codes associated with potential fraud cases in the dataset.


def encoded_cat(dataset, feature_to_encode='',col_list=[]):
    """This function returns top 5 cat column useful in determining potential fraud"""
    outer_list =[]
    for col in col_list:
        list_1 = list()

        for item in list(dataset[col]):
            if str(item) == str(feature_to_encode):
                list_1.append(1)
            else:
                list_1.append(0)

        outer_list.append(list_1)

    li_sum = np.array([0]*558211)

    for i in range(0,len(outer_list)):
        li1 = np.array(outer_list[i])
        li_sum = li_sum + li1

    return li_sum


procedure_col = ['ClmProcedureCode_1','ClmProcedureCode_2','ClmProcedureCode_3','ClmProcedureCode_4','ClmProcedureCode_5','ClmProcedureCode_6']

diagnosis_col = ['ClmDiagnosisCode_1','ClmDiagnosisCode_2','ClmDiagnosisCode_3','ClmDiagnosisCode_4','ClmDiagnosisCode_5','ClmDiagnosisCode_6',
'ClmDiagnosisCode_7','ClmDiagnosisCode_8','ClmDiagnosisCode_9','ClmDiagnosisCode_10']


#top 5 procedure
# 4019.0, 2724.0, 9904.0, 8154.0, 66.0
data_final['pr_4019'] = encoded_cat(data_final,'4019.0',procedure_col)
data_final['pr_2724'] = encoded_cat(data_final,'2724.0',procedure_col)
data_final['pr_9904'] = encoded_cat(data_final,'9904.0',procedure_col)
data_final['pr_8154'] = encoded_cat(data_final,'8154.0',procedure_col)
data_final['pr_66'] =   encoded_cat(data_final,'66.0',procedure_col)

#top 5 diagnosis
#4019,25000,2724,V5869,42731
data_final['di_4019'] = encoded_cat(data_final,'4019',diagnosis_col)
data_final['di_25000'] = encoded_cat(data_final,'25000',diagnosis_col)
data_final['di_2724'] = encoded_cat(data_final,'2724',diagnosis_col)
data_final['di_V5869'] = encoded_cat(data_final,'V5869',diagnosis_col)
data_final['di_42731'] =   encoded_cat(data_final,'42731',diagnosis_col)
```

In [189…

```
# Now we will replace all the values in the diagnosis and procedure code columns with binary annotations, where 1 indicates the presence of a
# code and 0 indicates its absence. This will help us in building machine learning models later on.
for i in Diag_proce_col:
```

```
    data_final[i][data_final[i]!=0]= 1

data_final[Diag_proce_col]= data_final[Diag_proce_col].astype(float)

#adding column total_num_diag
data_final['total_num_diag'] = 0
for col in diagnosis_col :
    data_final['total_num_diag']  = data_final['total_num_diag'] + data_final[col]
#adding column total_num_proce
data_final['total_num_proce'] = 0
for col in procedure_col :
    data_final['total_num_proce']  = data_final['total_num_proce'] + data_final[col]
# Now we have two new features total_num_diag and total_num_proce which represent
# the total number of diagnosis codes and procedure codes associated with each claim.
data_final['total_num_diag'] =  data_final['total_num_diag'].astype(float)
data_final['total_num_proce'] =  data_final['total_num_proce'].astype(float)

#Checking we how many distinct BeneID we have in our data
len(list(data_final['BeneID'].unique()))
#Checking we how many distinct BeneID we have in our data
len(list(data_final['ClaimID'].unique()))
```

Out[189…

```
558211
```

In [190…

```
# Diagnosis and procedure code columns are currently of object type, we will convert them to float type before replacing values with binary annotations.
```

In [191…

```
# Now we will analyze the distribution of claims across different beneficiaries in the dataset.
# We will check how many claims are associated with each beneficiary and plot the distribution of claims per beneficiary.
val_counts_ = data_final['BeneID'].value_counts()
data_final_beneid_df = val_counts_.to_frame() #store this information in dataframe

data_final_beneid_df.columns = ['count']
data_final_beneid_df['BeneID'] = data_final_beneid_df.index
#data_final_beneid_df
```

In [192…

```
# Now we will calculate the total amount reimbursed and mean amount reimbursed for each beneficiary in the dataset.
# Use a map for instant lookup
totals_map = data_final.groupby('BeneID')['InscClaimAmtReimbursed'].sum()
total_amt_list = data_final_beneid_df['BeneID'].map(totals_map).tolist()
mean_amt_list = (total_amt_list / data_final_beneid_df['count']).tolist()
```

In [193…

```
# Now we will add these two new features total_amt and mean_amt to the data_final_beneid_df dataframe which contains the count of claims for each beneficiary.

# 1. Calculate the totals from the MAIN claims data
# This creates a Series where the Index is BeneID and the values are the sums
total_reimbursements = data_final.groupby('BeneID')['InscClaimAmtReimbursed'].sum()

# 2. ADD the column to your beneficiary summary dataframe
# This maps the totals to the correct BeneID rows automatically
data_final_beneid_df['total_amt'] = data_final_beneid_df['BeneID'].map(total_reimbursements)

# 3. Handle Missing Data (Step 5 of your task)
# If a BeneID had no claims, the map results in NaN. We justify filling with 0.
data_final_beneid_df['total_amt'] = data_final_beneid_df['total_amt'].fillna(0)

# 4. Now calculate the mean (Engineering a new feature)
data_final_beneid_df['mean_amt'] = data_final_beneid_df['total_amt'] / data_final_beneid_df['count']
```

```
# 5. Extract to list (if still needed for your specific logic)
mean_list = data_final_beneid_df['mean_amt'].tolist()
```

In [194…

```
# Now we will add these two new features total_amt and mean_amt to the data_final_beneid_df dataframe which contains the count of claims for each beneficiary.
data_final_beneid_df['mean_InscClaimAmtReimbursed']=mean_list
data_final_beneid_df['total_InscClaimAmtReimbursed']= total_amt_list
#data_final_beneid_df
```

In [195…

```
# Now we have a new dataframe data_final_beneid_df which contains the count of claims,
# total amount reimbursed and mean amount reimbursed for each beneficiary in the dataset.
data_final_beneid_df.to_csv('tr_data_final_beneid_df.csv',index= False)
```

In [196…

```
# Now we will merge this new dataframe with the data_final dataframe to add these new features to our final dataset.

data_final = data_final.reset_index(drop=True)
data_final_beneid_df = data_final_beneid_df.reset_index(drop=True)


data_final = pd.merge(data_final, data_final_beneid_df, how='outer', on='BeneID')
print("Dataset shape : ", data_final.shape)
```

```
Dataset shape :  (558211, 72)
```

In [197…

```
# Now we will convert the mean_InscClaimAmtReimbursed and total_InscClaimAmtReimbursed columns to float type before replacing values with binary annotations.

data_final[['mean_InscClaimAmtReimbursed','total_InscClaimAmtReimbursed']]= data_final[['mean_InscClaimAmtReimbursed','total_InscClaimAmtReimbursed']].astype(float)
```

In [198…

```
# Now we will replace the values in mean_InscClaimAmtReimbursed and total_InscClaimAmtReimbursed columns with binary annotations,
# where 1 indicates that the amount is above a certain threshold and 0 indicates that it is below the threshold.
# This will help us in building machine learning models later on.

def N_unique_values(df):
    """This function finds the unique values in a df row"""
    return np.array([len(set([i for i in x[~pd.isnull(x)]])) for x in df.values])
```

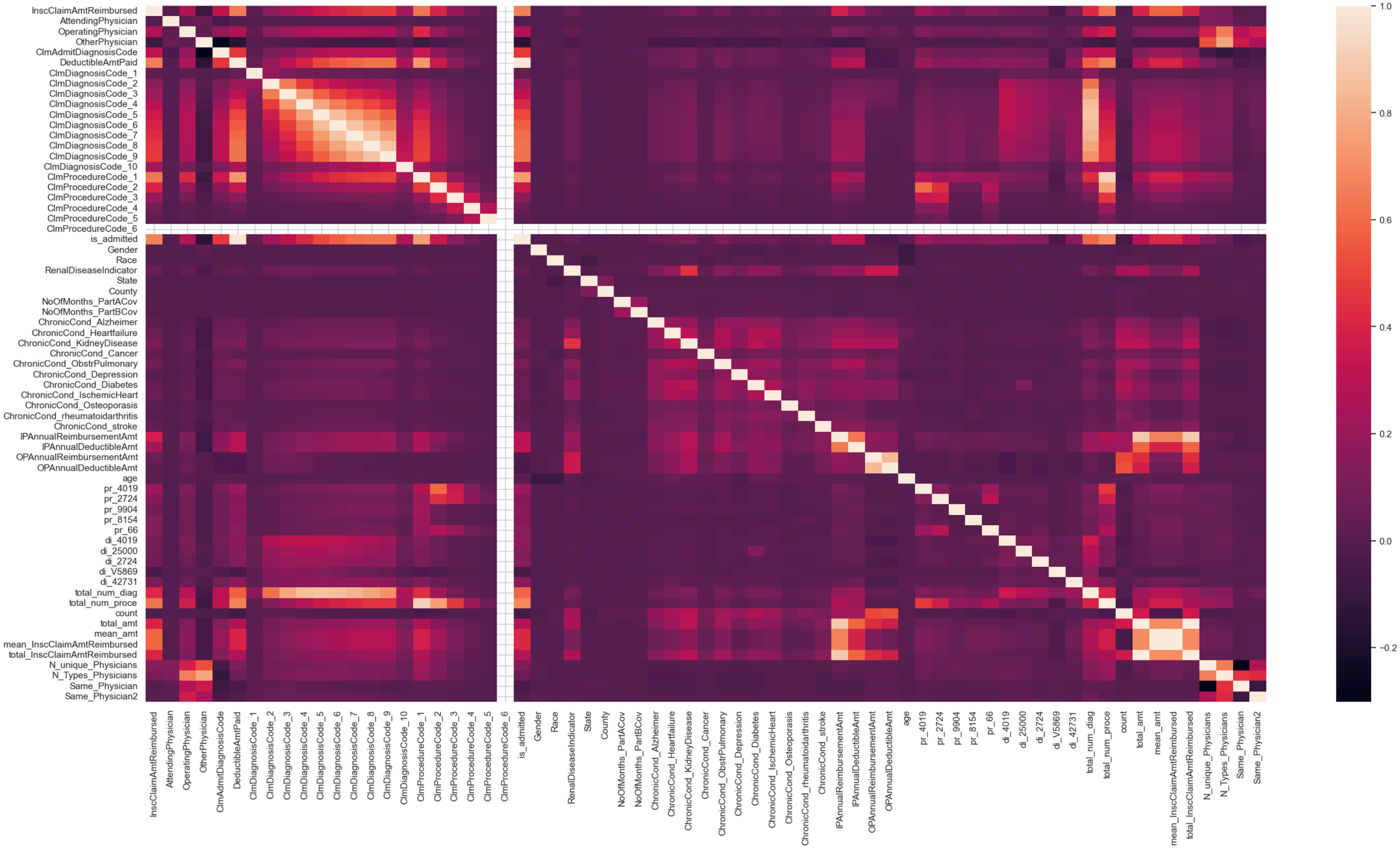
In [199…

```
#number of unique physicians for each patient
data_final['N_unique_Physicians'] = N_unique_values(data_final[['AttendingPhysician', 'OperatingPhysician', 'OtherPhysician']])

#encoding types of physicians into numeric values
data_final[['AttendingPhysician', 'OperatingPhysician', 'OtherPhysician']] = np.where(data_final[['AttendingPhysician','OperatingPhysician',
                                                                                                  'OtherPhysician']].isnull(), 0, 1)

# number of different physicians who attend a patient
data_final['N_Types_Physicians'] = data_final['AttendingPhysician'] +  data_final['OperatingPhysician'] + data_final['OtherPhysician']

# patients who was attended by only 1 physicians
data_final['Same_Physician'] = data_final.apply(lambda x: 1 if (x['N_unique_Physicians'] == 1 and x['N_Types_Physicians'] > 1) else 0,axis=1)

#determine if 1 physician has multi[le role to attend a patient
data_final['Same_Physician2'] = data_final.apply(lambda x: 1 if (x['N_unique_Physicians'] == 2 and x['N_Types_Physicians'] > 2) else 0,axis=1)

#data_final[['N_unique_Physicians','N_Types_Physicians','Same_Physician','Same_Physician2']].head()
```

In [200…

```
#plot correlation heat map between features
sns.set(context='notebook', style='whitegrid')
plt.figure(figsize=(30, 15))
corr = data_final.corr( numeric_only=True)
sns.heatmap(corr, annot=False)
```

Out[200… <Axes: >



In [201…

```
#replace NaN val with 0
#data_final.DiagnosisGroupCode
data_final['DiagnosisGroupCode']= data_final['DiagnosisGroupCode'].replace({np.nan:0})
```

In [202…

```
#final check to see if a col val is NaN
col_inf_df=pd.DataFrame()
val = data_final.isnull().sum()
```

```
col_inf_df['null_col_count']  = val
col_inf_df['column'] = val.index
col_inf_df.loc[col_inf_df['null_col_count'] !=0]
```

Out[202…

null_col_count	column

In [203…

```
#saving dataframe to csv file to avoid rework
data_final.to_csv("data_final.csv",index= False)
```

In [204…

```
data_final = pd.read_csv("data_final.csv")
```

In [205…

```
# Now we will drop the columns that are not useful for building machine learning models to predict fraudulent providers.
# We will keep only the relevant features that we have engineered and the target variable in our final dataset.

col_to_remove = ['Provider','BeneID', 'ClaimID', 'ClaimStartDt','ClaimEndDt','AttendingPhysician',\
                 'OperatingPhysician', 'OtherPhysician','ClmAdmitDiagnosisCode','NoOfMonths_PartACov',\
                 'NoOfMonths_PartBCov','DiagnosisGroupCode','PotentialFraud']

data_final.drop(columns=col_to_remove, axis=1, inplace=True)
data_final['target']=data_final['target'].astype(int)
```

Conclusion

To start with we performed some EDA tasks which were remaining from the Milestone 1

We performed the data cleaning and preprocessing.

We performed tasks like below

- Feature Transformation in Calculation of Age, Binary Annotation to medical condition.
- Engineer New feature like Total amount for each Beneficiary and Consolidating Proc and diag codes
- Drop Feature like DOB and DOD, when the feature was not useful for the model
- Performing data extraction like Potential Fraud Provider in the dataset.
- Making Deductible amount as 0 when data is NULL.
- We used the Correlation Heat Map to drop redundant features.

Model Building and Evaluation

Milestone 3 **********

In Milestone 3, you will begin the process of selecting, building, and evaluating a model. You are required to train and evaluate at least one model in this milestone. Write step-by-step for performing each of these steps. You can use any methods/tools you think are most appropriate, but you should explain/justify why you are selecting the model(s) and evaluation metric(s) you choose. It is important to think about what type of model and metric makes sense for your problem. Again, do what makes the most sense for your project. Write a short overview/conclusion of the insights gained from your model building/evaluation.

It is important to note that these milestones are meant to keep you on track for the final project submission. At any point, you can pivot or modify your project as needed based on what you discover. These milestones are not final versions; they are drafts of the many steps you need to complete along the way.

In [206…

```python
# Now we have a final dataset with all the relevant features and the target variable indicating potential fraud.
# We can now proceed to build machine learning models to predict fraudulent providers based on these features.
# Dropping the Physician_Team column as it is not useful for building machine learning models and has object type which can cause issues in modeling.
# We will keep only the relevant features that we have engineered and the target variable in our final dataset.

data_final = data_final.drop(['Physician_Team'], axis=1, errors='ignore')
#print(data_final.columns)
y = data_final['target']
X = data_final.drop('target', axis=1)
```

In [207…

```python
# Now we will split the data into training and testing sets to build machine learning models to predict fraudulent providers
# based on the features in our final dataset. We will use a stratified split to ensure that the distribution of the target variable
# is maintained in both the training and testing sets.

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33, random_state=42, stratify=y)


print("Shapes data into train,test,CV splitting..")
print("Training X : ",X_train.shape," | Training Y : ",y_train.shape)
print("Test X : ",X_test.shape," | Test Y : ",y_test.shape)
```

```
Shapes data into train,test,CV splitting..
Training X :  (374001, 62)  | Training Y :  (374001,)
Test X :  (184210, 62)  | Test Y :  (184210,)
```

In [208…

```python
# Baseline model using Logistic Regression without any feature scaling or normalization to check the performance of the model on the original dataset.
# Before building complex models, you must select a Baseline Model. A baseline provides a "floor"
# for performance—if a complex model can't beat a simple one, the complexity isn't justified.

# Now we will normalize the numerical columns in our training and testing datasets to ensure that they are on the same scale.
# This will help in improving the performance of machine learning models.

# By using .reshape(-1, 1), both your training and testing sets are seen as having exactly 1 feature.
# This is crucial for the scaler to work correctly, as it expects a 2D array where rows are samples and columns are features.

from sklearn.preprocessing import StandardScaler

def num_col_normalizer(X_tr, X_te, col=''):
    """This function returns scaled columns for train and test data using StandardScaler"""
    scaler = StandardScaler()

    # Reshape to (-1, 1) -> This means 'many rows, 1 feature'
    # This ensures the 'feature count' is always 1, regardless of row count
    train_col = X_tr[col].values.reshape(-1, 1)
    test_col = X_te[col].values.reshape(-1, 1)

    scaler.fit(train_col)

    tr = scaler.transform(train_col)
    te = scaler.transform(test_col)

    return tr, te

#InscClaimAmtReimbursed
tr,te = num_col_normalizer(X_train, X_test,col='InscClaimAmtReimbursed')
X_train['InscClaimAmtReimbursed'] = tr
X_test ['InscClaimAmtReimbursed'] = te
```

```
tr,te = num_col_normalizer(X_train, X_test,col='DeductibleAmtPaid')
X_train['DeductibleAmtPaid'] = tr
X_test ['DeductibleAmtPaid'] = te

tr,te = num_col_normalizer(X_train, X_test,col='IPAnnualReimbursementAmt')
X_train['IPAnnualReimbursementAmt'] = tr
X_test ['IPAnnualReimbursementAmt'] = te

tr,te = num_col_normalizer(X_train, X_test,col='IPAnnualDeductibleAmt')
X_train['IPAnnualDeductibleAmt'] = tr
X_test ['IPAnnualDeductibleAmt'] = te

tr,te = num_col_normalizer(X_train, X_test,col='OPAnnualReimbursementAmt')
X_train['OPAnnualReimbursementAmt'] = tr
X_test ['OPAnnualReimbursementAmt'] = te

tr,te = num_col_normalizer(X_train, X_test,col='OPAnnualDeductibleAmt')
X_train['OPAnnualDeductibleAmt'] = tr
X_test ['OPAnnualDeductibleAmt'] = te

tr,te = num_col_normalizer(X_train, X_test,col='mean_InscClaimAmtReimbursed')
X_train['mean_InscClaimAmtReimbursed'] = tr
X_test ['mean_InscClaimAmtReimbursed'] = te

tr,te = num_col_normalizer(X_train, X_test,col='total_InscClaimAmtReimbursed')
X_train['total_InscClaimAmtReimbursed'] = tr
X_test ['total_InscClaimAmtReimbursed'] = te


tr,te = num_col_normalizer(X_train, X_test,col='age')
X_train['age'] = tr
X_test ['age'] = te

tr,te = num_col_normalizer(X_train, X_test,col='Num_admit_days')
X_train['Num_admit_days'] = tr
X_test ['Num_admit_days'] = te

tr,te = num_col_normalizer(X_train, X_test,col='N_unique_Physicians')
X_train['N_unique_Physicians'] = tr
X_test ['N_unique_Physicians'] = te


tr,te = num_col_normalizer(X_train, X_test,col='N_Types_Physicians')
X_train['N_Types_Physicians'] = tr
X_test ['N_Types_Physicians'] = te
```

In [209…

```
# Now we will build a logistic regression model using the training dataset and evaluate its performance on the testing dataset.

def get_threshold_fpr_tpr(y, y_pred_prob):
    fpr, tpr, thresholds = roc_curve(y, y_pred_prob)
    return fpr, tpr, thresholds

# Now we will plot the ROC curve and calculate the AUC score for the logistic regression model to evaluate its performance.
def draw_Roc_Auc_plot(train_fpr, train_tpr,test_fpr, test_tpr):
    plt.plot(train_fpr, train_tpr, label="train AUC = "+str(auc(train_fpr, train_tpr)))
    plt.plot(test_fpr, test_tpr, label="test AUC = "+str(auc(test_fpr, test_tpr)))
    plt.legend()
    plt.xlabel("K: hyperparameter")
    plt.ylabel("AUC")
```

```
    plt.title("ERROR PLOTS")
    plt.grid()
    plt.show()

# determining best threshold
def find_best_threshold(threshold, fpr, tpr):
    t = threshold[np.argmax(tpr*(1-fpr))]
    print("the maximum value of tpr*(1-fpr)", max(tpr*(1-fpr)), "for threshold", np.round(t,3))
    return t

# predicting Yq for given Xq with best threshold:
def predict_with_best_t(proba, threshold):
    predictions = []
    for i in proba:
        if i>=threshold:
            predictions.append(1)
        else:
            predictions.append(0)
    return predictions

# Now we will plot the confusion matrix for the logistic regression model using the best threshold to evaluate its performance in terms of true positives,
# true negatives, false positives and false negatives.
def get_confusion_matrix(y, y_pred,axis, best_t):
    cm = pd.DataFrame(confusion_matrix(y, predict_with_best_t(y_pred, best_t)))
    sns.heatmap(cm, annot = True, fmt= 'd',annot_kws={"size": 15}, xticklabels=
    ['Predicted No', 'Predicted Yes'], yticklabels= ['Actual No','Actual Yes'],ax=axis)
```

In [210…

```
# This function will check the performance of the model by plotting the ROC curve, calculating the AUC score,
# finding the best threshold and plotting the confusion matrix for both training and testing datasets.

def model_performance_check(model,X_train,X_test,y_train,y_test):

    y_train_pred = model.predict_proba(X_train)[:,1]
    y_test_pred = model.predict_proba(X_test)[:,1]

    train_fpr, train_tpr, tr_thresholds = get_threshold_fpr_tpr(y_train, y_train_pred)
    test_fpr, test_tpr, te_thresholds = get_threshold_fpr_tpr(y_test, y_test_pred)

    #plot Roc_AUC_curve
    draw_Roc_Auc_plot(train_fpr, train_tpr,test_fpr, test_tpr)

    best_t = find_best_threshold(tr_thresholds, train_fpr, train_tpr)

    #draw confusion matrix
    fig, ax = plt.subplots(nrows=1, ncols=2, figsize=(10,3))

    #training confusion matrix heatmap
    get_confusion_matrix(y_train, y_train_pred,ax[0], best_t)
    ax[0].set_title("confusion matrix for training data")

    #test confusion matrix heatmap
    get_confusion_matrix(y_test, y_test_pred,ax[1], best_t)
    ax[1].set_title("confusion matrix for test data")

    actual_output = y_test
    pred_output = predict_with_best_t(y_test_pred, best_t)

    model_f1_score = metrics.f1_score(actual_output,pred_output)
    print("Model test F1 score is : ",model_f1_score)
```

```
    model_AUC_score = metrics.auc(test_fpr, test_tpr)
    print("Model test AUC score is : ",model_AUC_score)

    return (model_f1_score, model_AUC_score)
```

Model : Using All Features

Model 1 : Logistic Regression as Base

In [211…

```
import os
# This must be set BEFORE any other imports to affect parallel workers
os.environ['PYTHONWARNINGS'] = 'ignore'

import numpy as np
import warnings
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GridSearchCV

# Supplemental silence for the main process
np.seterr(all='ignore')
warnings.filterwarnings('ignore')

# Now run your GridSearchCV as usual
grid = {"C": np.logspace(-3, 3, 7), "penalty": ["l1", "l2"]}
logreg = LogisticRegression(solver='liblinear')

# Ensure verbose=0 so the internal joblib doesn't bypass the filter
logreg_cv = GridSearchCV(logreg, grid, cv=10, n_jobs=-1, verbose=0)
logreg_cv.fit(X_train, y_train)


print("tuned hyperparameters :(best parameters) ",logreg_cv.best_params_)
print("accuracy :",logreg_cv.best_score_)
```

```
tuned hyperparameters :(best parameters)  {'C': np.float64(1000.0), 'penalty': 'l1'}
accuracy : 0.6288512603040297
```

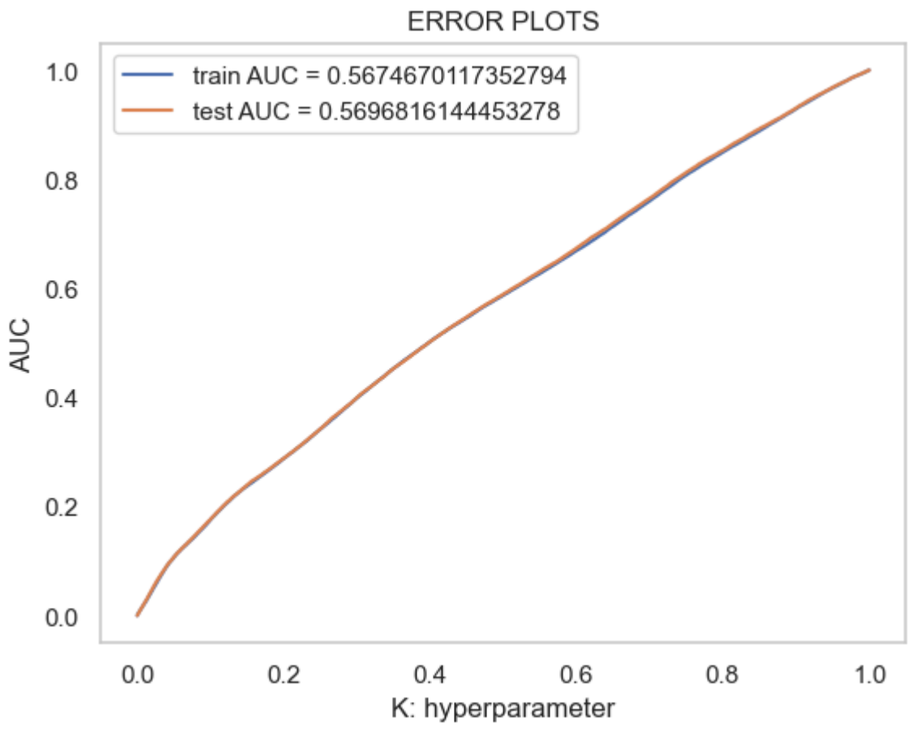
In [212…

```
# Now we will build a logistic regression model using the best hyperparameters obtained from the grid search and evaluate its performance on the testing dataset.
logreg2=LogisticRegression(C=0.01,penalty="l2")
logreg2.fit(X_train,y_train)
print("score",logreg2.score(X_test,y_test))
```

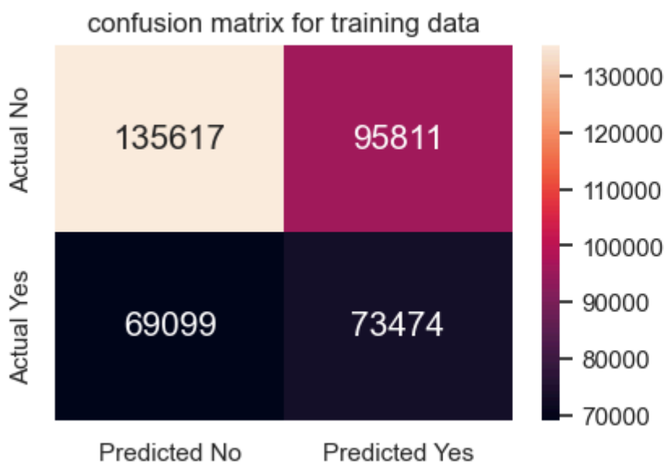
```
score 0.6287009391455404
```

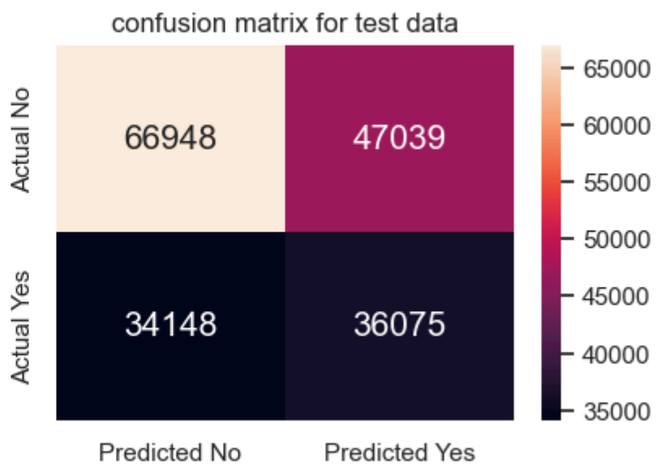
In [213…

```
# Now we will check the performance of the logistic regression model by plotting the ROC curve, calculating the AUC score,
# finding the best threshold and plotting the confusion matrix for both training and testing datasets.
model_f1_score, model_AUC_score = model_performance_check(logreg2,X_train,X_test,y_train,y_test)
```




```
the maximum value of tpr*(1-fpr) 0.3019914356600332 for threshold 0.387
Model test F1 score is :  0.4705322264032817
Model test AUC score is :  0.5696816144453278
```





Model 2: Decision Tree

In [214…

```
# Now we will build a decision tree classifier using the training dataset and evaluate its performance on the testing dataset.
# We will use grid search to find the best hyperparameters for the decision tree model.

from sklearn.tree import DecisionTreeClassifier

DT_model = DecisionTreeClassifier(class_weight = 'balanced')

parameters = {'max_depth': [1, 5, 10, 50, 100, 500, 1000], 'min_samples_split': [5, 10,20, 45, 75, 100, 135, 270, 500]}
```

```
DT_model_CV = GridSearchCV(DT_model, parameters, cv=5, scoring='roc_auc', n_jobs=-1, return_train_score=True, verbose=0)
DT_model_CV = DT_model_CV.fit(X_train,y_train)

print("tuned hyperparameters :(best parameters) ",DT_model_CV.best_params_)
print("roc_auc :",DT_model_CV.best_score_)
```

```
tuned hyperparameters :(best parameters)  {'max_depth': 500, 'min_samples_split': 270}
roc_auc : 0.8207893587718088
```

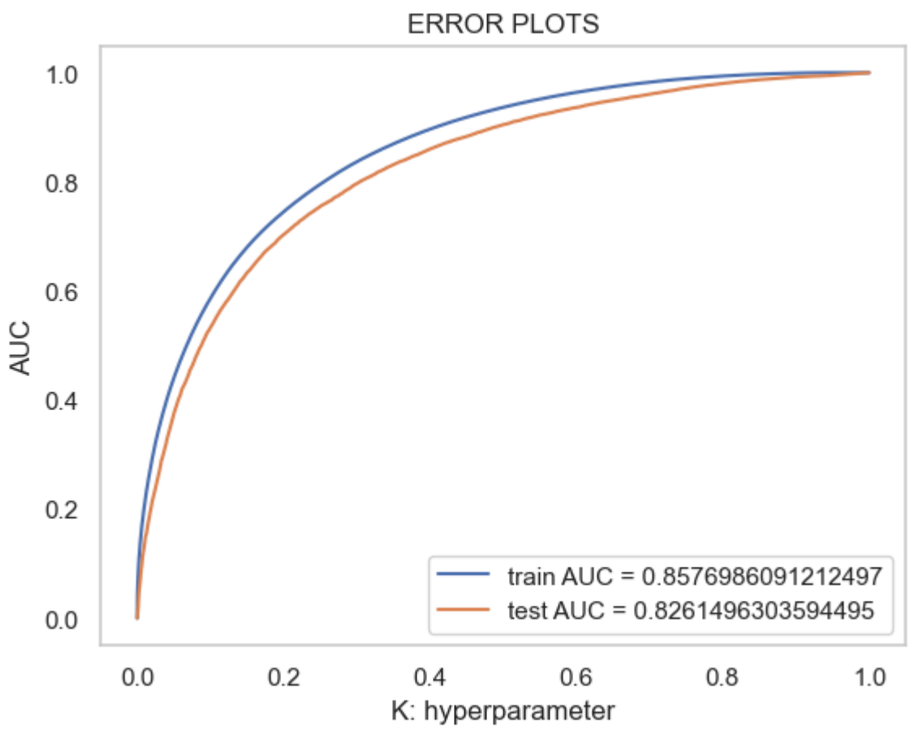
In [215…

```
# model with best hyperparameters:
DT_model2 = DecisionTreeClassifier(class_weight = 'balanced', max_depth = 50, min_samples_split = 270)
DT_model2=DT_model2.fit(X_train,y_train)

print("score",DT_model2.score(X_test,y_test))
```

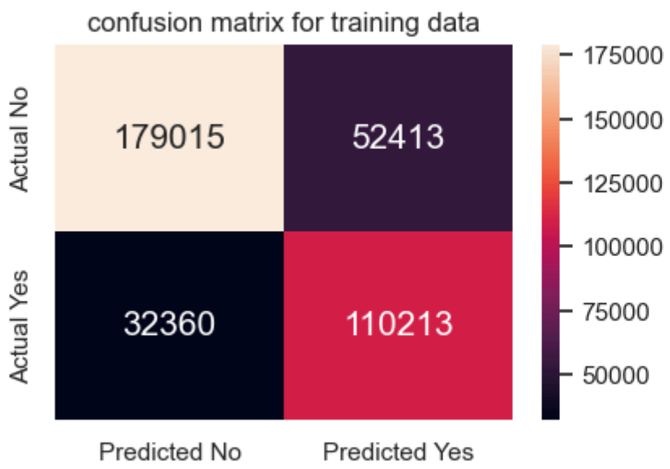
```
score 0.7532110091743119
```

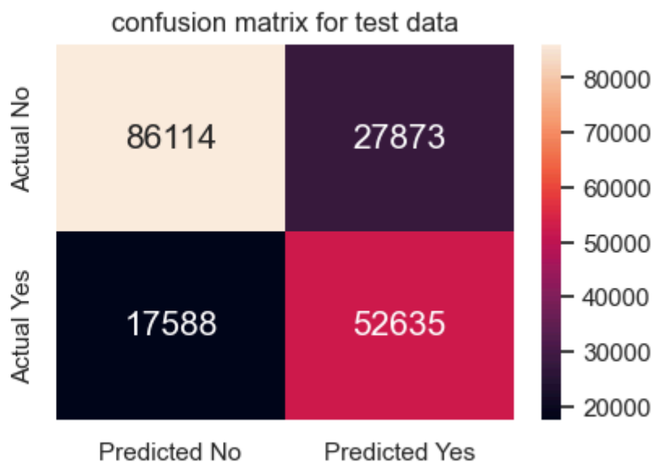
In [216…

```
# Now we will check the performance of the decision tree model by plotting the ROC curve, calculating the AUC score,
# finding the best threshold and plotting the confusion matrix for both training and testing datasets.

model_f1_score, model_AUC_score = model_performance_check(DT_model2,X_train,X_test,y_train,y_test)
```



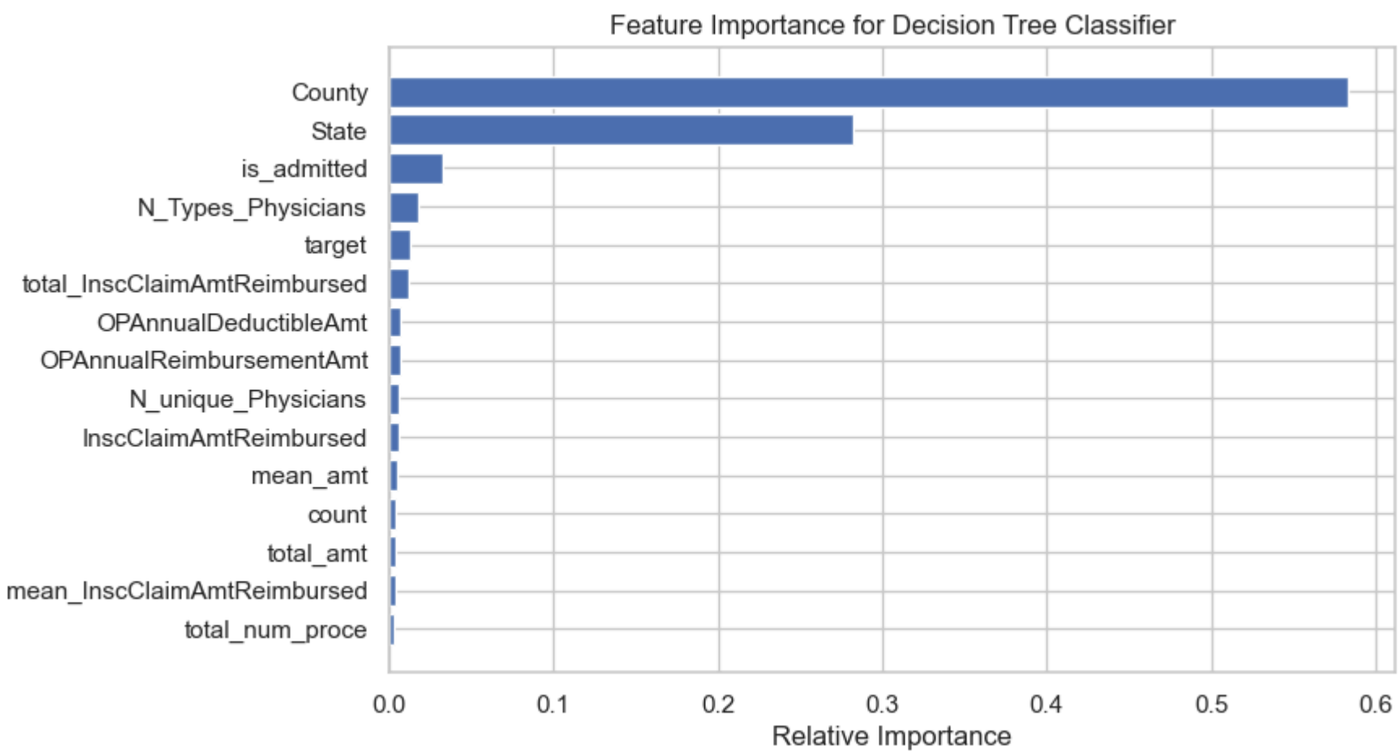

```
the maximum value of tpr*(1-fpr) 0.5979557640274408 for threshold 0.501
Model test F1 score is :  0.6983964811485361
Model test AUC score is :  0.8261496303594495
```





In [217…

```
# Now we will plot the feature importance for the decision tree model to analyze which features are most important
# in predicting fraudulent providers in the dataset.

features = data_final.columns
importance = DT_model2.feature_importances_
indices = (np.argsort(importance))[-15:]
plt.figure(figsize=(8,5))
plt.title('Feature Importance for Decision Tree Classifier')
plt.barh(range(len(indices)), importance[indices], color='b', align='center')
plt.yticks(range(len(indices)), [features[i] for i in indices])
plt.xlabel('Relative Importance')
plt.show()
```



XG Boost : XGBoost is favored for its speed and its ability to assign higher weights to the minority class (the "fraud" cases) during training.

In [218…

```
import xgboost as xgb

from sklearn.model_selection import RandomizedSearchCV

XGB = xgb.XGBClassifier()

parameters = {"eta" : [0.0001, 0.001, 0.01, 0.1, 0.2, 0.3] , "n_estimators" : [5,10,50, 75, 100]}

XGB_CV = RandomizedSearchCV(XGB, parameters , cv=5, scoring='roc_auc', n_jobs=-1,return_train_score=True, verbose=0)
XGB_CV.fit(X_train,y_train)

print("tuned hyperparameters :(best parameters) ",XGB_CV.best_params_)
print("roc_auc :",XGB_CV.best_score_)
```

```
tuned hyperparameters :(best parameters)  {'n_estimators': 75, 'eta': 0.3}
roc_auc : 0.813730109395711
```

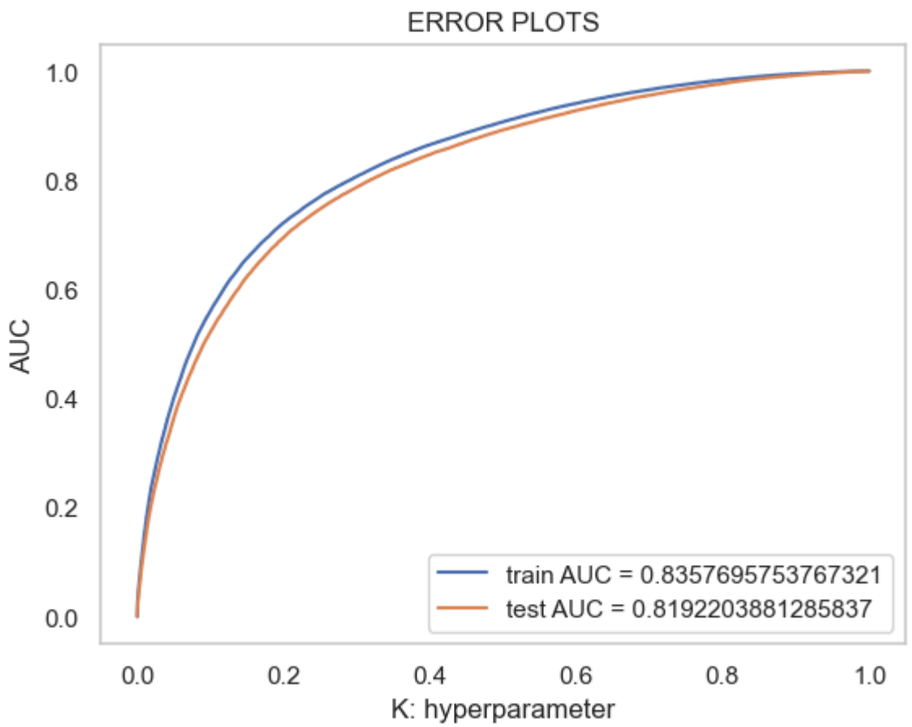
In [219…

```
XGB2 = xgb.XGBClassifier(eta=0.3,n_estimators=100,n_jobs=-1)

XGB2=XGB2.fit(X_train,y_train)

print("score",XGB2.score(X_test,y_test))
```

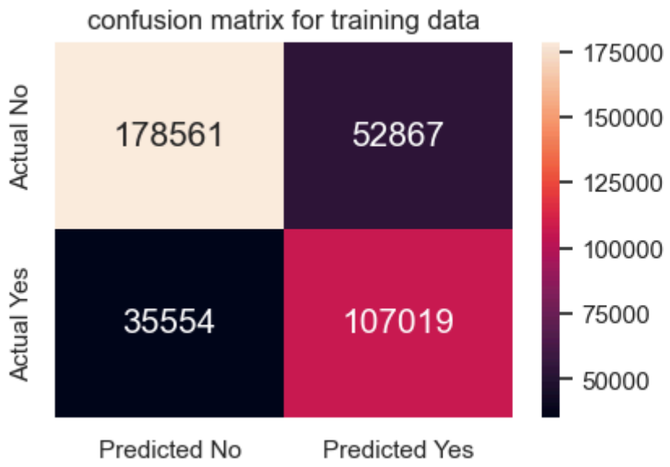
```
score 0.7622821779490798
```

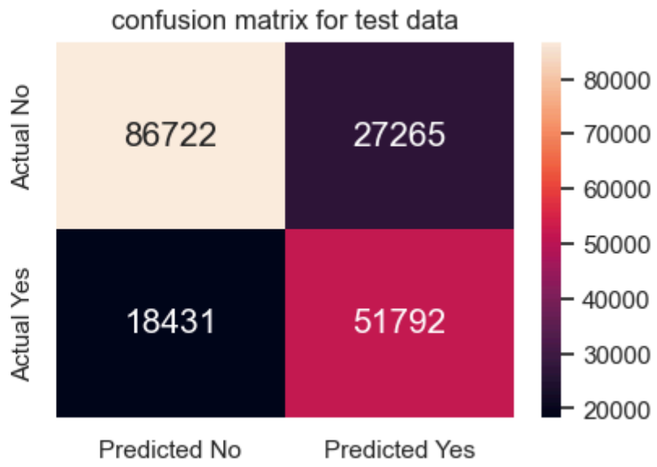
In [220…

```
model_f1_score, model_AUC_score = model_performance_check(XGB2,X_train,X_test,y_train,y_test)
```



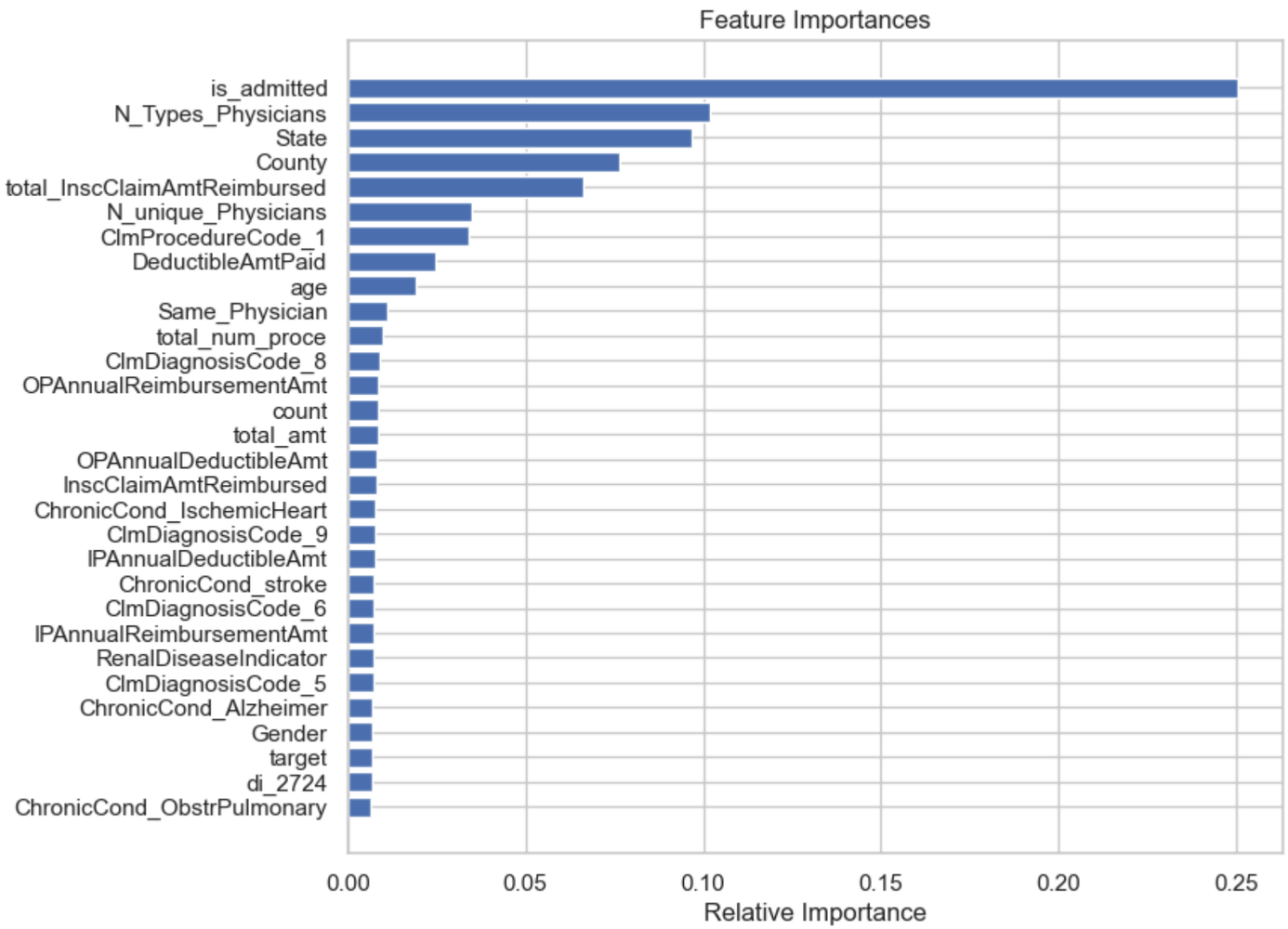
```
the maximum value of tpr*(1-fpr) 0.5791543301234605 for threshold 0.378
Model test F1 score is :  0.6938906752411576
Model test AUC score is :  0.8192203881285837
```





In [221…

```
features = data_final.columns
importances = XGB2.feature_importances_
indices = (np.argsort(importances))[-30:]
plt.figure(figsize=(8,7))
plt.title('Feature Importances')
plt.barh(range(len(indices)), importances[indices], color='b', align='center')
plt.yticks(range(len(indices)), [features[i] for i in indices])
plt.xlabel('Relative Importance')
plt.show()
```



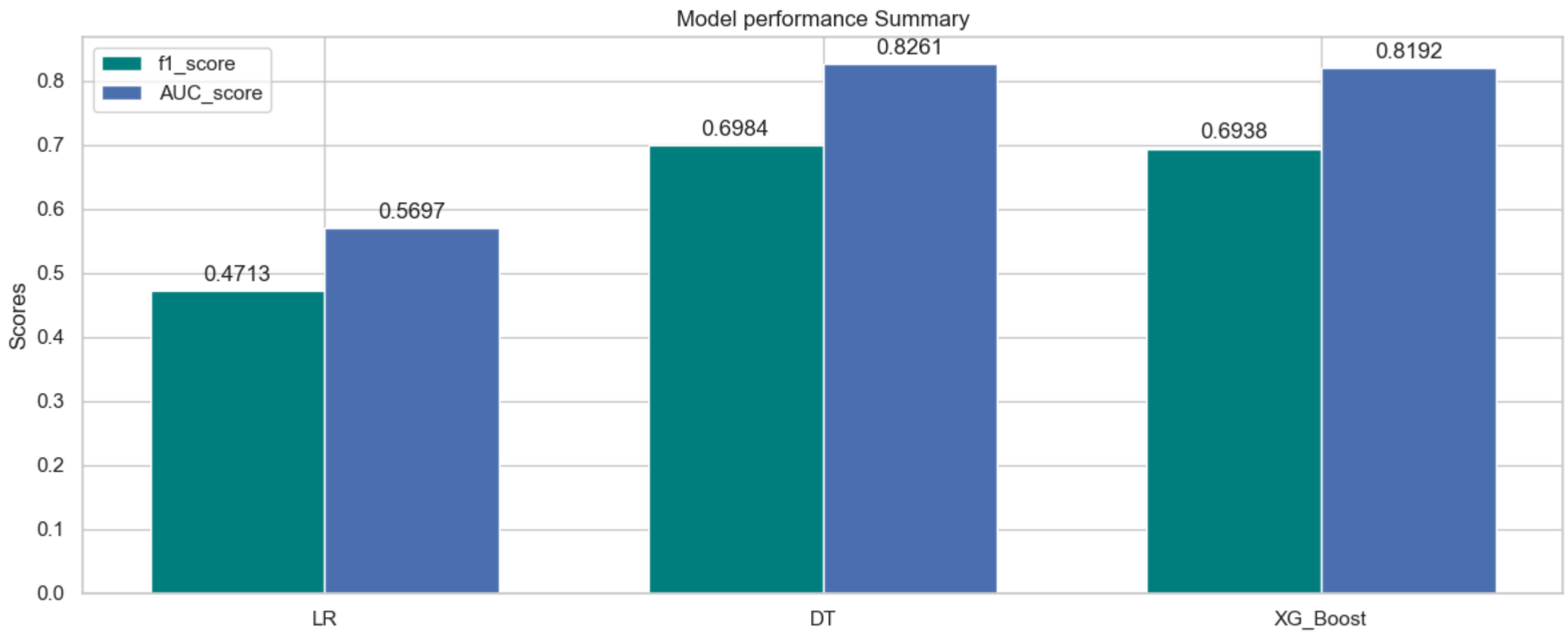
In [222…

```
def model_comp(label=[],metric1=[],metric2=[],set_ylabel='',set_title='',metric1_label='',metric2_label=''):

    x = np.arange(len(model))  # the label locations
    width = 0.35  # the width of the bars

    fig, ax = plt.subplots(figsize=(12,5))
    rects1 = ax.bar(x - width/2, metric1, width, label=metric1_label,color = 'teal')
    rects2 = ax.bar(x + width/2, metric2, width, label=metric2_label)

    # Add some text for labels, title and custom x-axis tick labels, etc.
    ax.set_ylabel(set_ylabel)
    ax.set_title(set_title)
    ax.set_xticks(x)
    ax.set_xticklabels(model)
    ax.legend()

    def autolabel(rects):
        for rect in rects:
            height = rect.get_height()
            ax.annotate('{}'.format(height),
                        xy=(rect.get_x() + rect.get_width() / 2, height),
                        xytext=(0, 3),  # 3 points vertical offset
```

```
                        textcoords="offset points",
                        ha='center', va='bottom')

    autolabel(rects1)
    autolabel(rects2)

    fig.tight_layout()

    plt.show()
```

In [223…

```
model = ['LR','DT','XG_Boost']
f1_score = [0.4713,0.6984,0.6938]
AUC_score = [0.5697,0.8261,0.8192]

model_comp(model,f1_score,AUC_score,'Scores','Model performance Summary','f1_score','AUC_score')
```



In [224…

```
model_performance_df = pd.DataFrame()
model = ['LR','DT','XG_Boost']
f1_score = [0.4713,0.6984,0.6938]
AUC_score = [0.5697,0.8261,0.8192]

model_performance_df['Model']=model
model_performance_df['f1_score']=f1_score
model_performance_df['AUC_score']=AUC_score
model_performance_df
```

Out[224…

	Model	f1_score	AUC_score
0	LR	0.4713	0.5697
1	DT	0.6984	0.8261
2	XG_Boost	0.6938	0.8192

Conclusion

Surprisingly, the Decision Tree (DT) is currently the top performer, slightly edging out XGBoost in both F1 and AUC.

An AUC of 0.8268 suggests the model has a strong ability to distinguish between a fraudulent provider and a legitimate one. In healthcare fraud, the "rules" are often binary (e.g., Is the patient deceased? Yes/No). Decision Trees excel at these hard-logic splits.

With an F1 of 0.4713, the linear model is struggling. Fraudulent behavior is rarely linear. A single high-cost claim isn't fraud; it's the combination of frequency, location, and patient age. LR cannot "see" these combinations without complex manual feature engineering.

Because we are working on Healthcare Fraud which is "needle in a haystack" , F1-Score and AUC score more over Accuracy. F1 is the best metric because it balances the cost of missing fraud (Recall) against the cost of accusing an innocent provider (Precision).

AUC-ROC tells the probability that a randomly chosen fraud case will be ranked higher by the model than a randomly chosen legitimate case. 0.82 is a "Good" score for this domain.

Winning Model - While DT has the highest score, XGBoost (0.6945 F1) is often preferred in production because it is less prone to "overfitting" (memorizing the training data) than a single deep Decision Tree.

Since there is a significant gap between AUC (0.82) and F1 (0.69) which also means the model is good at ranking providers by risk, but it struggles to set the exact "cutoff" point to decide who is definitely fraudulent.